

Sumário

Reconstruir Docker	Erro! Indicador não definido.
Para rodar o frontend atualizado da Main	Erro! Indicador não definido.
Rodar Browser	Erro! Indicador não definido.
Seleção de casos 3.....	2
Cenário 1: Degradação de Latência (degradation).....	2
Teste 1.1: O Exemplo Canônico (Amplificação de Carga)	2
Teste 1.2: Risco de Cascata Oculto (Sem Timeout)	4
Teste 1.3: Degradação Segura (Absorvida).....	6
Cenário 2: Indisponibilidade Total (unavailability).....	8
Teste 2.1: SPOF Síncrono Fatal	8
Teste 2.2: Isolamento por Assincronia (Blast Radius Contido)	10
Teste 2.3: Efeito Dominó de Retries	12
Cenário 3: Multiplicador de Carga (load_multiplier)	14
Teste 3.1: Black Friday (Saturação Massiva).....	14
Teste 3.2: Pico Absorvido	16
Teste 3.3: Gargalo no Banco (Assimetria de Escala).....	18
Cenário 4: Validações de Borda (Adversariais e Fora de Escopo)	20
Teste 4.1: Descrição Incompleta (Bloqueante).....	20
Teste 4.2: Pergunta Fora de Escopo	21
Teste 4.3: Texto Contraditório (Engenheiro no Slack)	22

Seleção de casos 3

Casos de Teste (Golden Tests & Validation Suite)

Abaixo estão 3 casos para cada cenário. Eles servem como seus **Golden Tests** (deriváveis analiticamente à mão) e cobrem as exigências do *Validation Suite*.

Cenário 1: Degradação de Latência (degradation)

Teste 1.1: O Exemplo Canônico (Amplificação de Carga)

```
{
  "architecture_text": "A API de checkout chama a precificação de forma síncrona com timeout de 2s e 3 retries. A precificação lê do Postgres. Checkout tem 6 réplicas, precificação 4, lidando com 80 rps cada.",
  "question": "O que acontece se o Postgres degradar para 3s?" }
```

○ Gabarito Esperado:

- Latência acumulada > 2s (Timeout estourado).
- Amplificação de 4x (1 chamada original + 3 retries) na precificação.
- Raio de impacto: Checkout, Precificação.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "checkout_system",
    "status": "ready",
    "nodes": [
      {
        "id": "checkout",
        "replicas": 6,
        "max_rps_per_replica": 80,
        "timeout_ms": null
      },
      {
        "id": "pricing",
        "replicas": 4,
        "max_rps_per_replica": 80,
        "timeout_ms": null
      },
      {
        "id": "postgres",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "checkout",
        "target": "pricing",
        "call_type": "sync",
        "retries": 3,
        "backoff_ms": null,
        "timeout_ms": 2000
      },
      {
        "source": "pricing",
        "target": "postgres",
        "call_type": "sync",

```

```

    "retries": null,
    "backoff_ms": null,
    "timeout_ms": null
  }
],
"target_availability": 0.999,
"scenario": {
  "scenario_type": "degradation",
  "target_node": "postgres",
  "parameter_value": 3000,
  "reason": null
},
"assumptions_made": [
  "Assumido 1 réplica para postgres",
  "Assumida capacidade irrestrita para postgres (max_rps_per_replica=null)",
  "Assumido call_type sync para a conexão pricing->postgres",
  "Assumido retries null e backoff null para a conexão pricing->postgres"
],
"missing_critical_info": [],
"reason": null
},
"semantic_issues": [
  "Ponto Único de Falha (SPOF): O componente 'postgres' possui apenas 1 réplica.",
  "Amplificação de Carga: A chamada 'checkout' -> 'pricing' possui 3 retries que podem amplificar falha s.",
  "Risco de Cascata: A chamada síncrona de 'pricing' para 'postgres' não possui timeout configurado."
],
"simulation_results": {
  "scenario": "Degradação de latência no componente 'postgres' para 3000.0ms",
  "timeouts_exceeded": [
    "A chamada 'checkout' -> 'pricing' excede o timeout de 2000.0ms (Latência real: 3050.0ms)"
  ],
  "load_amplifications": [
    "As reexecuções do checkout amplificam a carga no pricing em 4x (Carga efetiva: 960.0 rps)"
  ],
  "node_saturation": {
    "checkout": "50.0%",
    "pricing": "75.0%",
    "postgres": "300.0%"
  },
  "blast_radius_impacted_nodes": [
    "checkout",
    "pricing"
  ],
  "availability_impact": "A disponibilidade cai abaixo da meta (99.9%) devido à saturação de postgres",
  "blast_radius_summary": "2 de 3 componentes impactados (checkout, pricing)"
},
"telemetry": {
  "latency_ms": 1983.26,
  "tokens_in": 1233,
  "tokens_out": 661,
  "total_tokens": 1894,
  "estimated_cost_usd": 0.00125
}
}

```

Response headers

Teste 1.2: Risco de Cascata Oculto (Sem Timeout)

{ "architecture_text": "O serviço de Autenticação chama o serviço de Perfil de Usuário de forma síncrona. Ambos rodaram com 2 réplicas suportando 50 rps cada. Não há retries configurados."
,"question": "Qual o impacto se o Perfil de Usuário sofrer lentidão de 10 segundos?"
}

- **Gabarito Esperado:**

- O validator.py deve apontar *Risco de Cascata* (falta de timeout).
- Não há amplificação de RPS (retries = 0).
- Disponibilidade é afetada pela exaustão de threads, não por volume de requisições.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Authentication System",
    "status": "ready",
    "nodes": [
      {
        "id": "auth",
        "replicas": 2,
        "max_rps_per_replica": 50,
        "timeout_ms": null
      },
      {
        "id": "user-profile",
        "replicas": 2,
        "max_rps_per_replica": 50,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "auth",
        "target": "user-profile",
        "call_type": "sync",
        "retries": 0,
        "backoff_ms": null,
        "timeout_ms": null
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "degradation",
      "target_node": "user-profile",
      "parameter_value": 10000,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 2 réplicas para auth",
      "Assumido 2 réplicas para user-profile",
      "Assumido sem retries configurados (retries=0)"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Risco de Cascata: A chamada síncrona de 'auth' para 'user-profile' não possui timeout configurado."
  ],
  "simulation_results": {
    "scenario": "Degradação de latência no componente 'user-profile' para 10000.0ms",
    "timeouts_exceeded": [],
    "load_amplifications": [],
    "node_saturation": {
      "auth": "50.0%",
      "user-profile": "50.0%"
    }
  }
}
```

```
    },
    "blast_radius_impacted_nodes": [],
    "availability_impact": "Dentro da meta",
    "blast_radius_summary": "0 de 2 componentes impactados ()"
  },
  "telemetry": {
    "latency_ms": 1672.09,
    "tokens_in": 1225,
    "tokens_out": 583,
    "total_tokens": 1808,
    "estimated_cost_usd": 0.001183
  }
}
```

Response headers

Teste 1.3: Degradação Segura (Absorvida)

"architecture_text": "O microserviço de Relatórios consulta o banco MySQL de forma síncrona com timeout de 15 segundos. Zero retries. Capacidade irrestrita."
, "question": "O que acontece se a latência do MySQL subir para 5 segundos?"}

Gabarito Esperado:

Nenhum timeout excedido (5s < 15s).

Nenhuma amplificação de carga.

Disponibilidade: Dentro da meta.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Report Service",
    "status": "ready",
    "nodes": [
      {
        "id": "reports",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "mysql",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "reports",
        "target": "mysql",
        "call_type": "sync",
        "retries": 0,
        "backoff_ms": null,
        "timeout_ms": 15000
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "degradation",
      "target_node": "mysql",
      "parameter_value": 5000,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para reports",
      "Assumida capacidade irrestrita para reports (max_rps_per_replica=null)",
      "Assumido 1 réplica para mysql",
      "Assumida capacidade irrestrita para mysql (max_rps_per_replica=null)"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Ponto Único de Falha (SPOF): O componente 'reports' possui apenas 1 réplica.",
    "Ponto Único de Falha (SPOF): O componente 'mysql' possui apenas 1 réplica."
  ],
  "simulation_results": {
    "scenario": "Degradação de latência no componente 'mysql' para 5000.0ms",
    "timeouts_exceeded": [],
    "load_amplifications": [],
    "node_saturation": {
```

```
    "reports": "50.0%",  
    "mysql": "50.0%"  
  },  
  "blast_radius_impacted_nodes": [],  
  "availability_impact": "Dentro da meta",  
  "blast_radius_summary": "0 de 2 componentes impactados ()"  
},  
"telemetry": {  
  "latency_ms": 1829.51,  
  "tokens_in": 1214,  
  "tokens_out": 654,  
  "total_tokens": 1868,  
  "estimated_cost_usd": 0.001233  
}  
}
```

Response headers

Cenário 2: Indisponibilidade Total (unavailability)

Teste 2.1: SPOF Síncrono Fatal

{"architecture_text": "O Gateway de Pagamentos roteia todas as requisições para uma única instância do serviço de Antifraude via chamada síncrona com 1 retry."}

,"question": "O que acontece se o serviço de Antifraude cair completamente?"}

Gabarito Esperado:

O validator.py detecta *Ponto Único de Falha (SPOF)*.

Amplificação de 2x no gateway (1 retry inútil).

Disponibilidade cai a zero no fluxo dependente.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Sistema de Pagamento",
    "status": "ready",
    "nodes": [
      {
        "id": "gateway-pagamentos",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "antifraude",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "gateway-pagamentos",
        "target": "antifraude",
        "call_type": "sync",
        "retries": 1,
        "backoff_ms": null,
        "timeout_ms": null
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "unavailability",
      "target_node": "antifraude",
      "parameter_value": null,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para gateway-pagamentos",
      "Assumido 1 réplica para antifraude",
      "Assumida capacidade irrestrita para gateway-pagamentos (max_rps_per_replica=null)",
      "Assumida capacidade irrestrita para antifraude (max_rps_per_replica=null)",
      "Assumida chamada síncrona (sync) entre gateway-pagamentos e antifraude"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Ponto Único de Falha (SPOF): O componente 'gateway-pagamentos' possui apenas 1 réplica.",
    "Ponto Único de Falha (SPOF): O componente 'antifraude' possui apenas 1 réplica.",
    "Risco de Cascata: A chamada síncrona de 'gateway-pagamentos' para 'antifraude' não possui timeout configurado."
  ]
}
```

```

    "Amplificação de Carga: A chamada 'gateway-pagamentos' -> 'antifraude' possui 1 retries que podem amplificar falhas."
  ],
  "simulation_results": {
    "scenario": "Falha total e indisponibilidade do componente 'antifraude'",
    "timeouts_exceeded": [
      "A chamada 'gateway-pagamentos' -> 'antifraude' falha imediatamente (Conexão Recusada / Timeout Imediato).",
    ],
    "load_amplifications": [
      "Sem resposta, os retries do 'gateway-pagamentos' amplificam a carga na rede em 2x (80.0 rps inúteis).",
    ],
    "node_saturation": {},
    "blast_radius_impacted_nodes": [
      "antifraude",
      "gateway-pagamentos"
    ],
    "availability_impact": "A disponibilidade do fluxo que depende de 'antifraude' cai a zero.",
    "blast_radius_summary": "2 de 2 componentes impactados no sistema (antifraude, gateway-pagamentos)"
  },
  "telemetry": {
    "latency_ms": 1954.12,
    "tokens_in": 1211,
    "tokens_out": 760,
    "total_tokens": 1971,
    "estimated_cost_usd": 0.001315
  }
}

```

Response headers

Teste 2.2: Isolamento por Assincronia (Blast Radius Contido)

{"architecture_text": "A API de Pedidos publica um evento no RabbitMQ. O Worker de Notificação consome essa fila de forma assíncrona."}

,"question": "O que acontece se o Worker de Notificação ficar offline?"}

Gabarito Esperado:

Nenhum timeout excedido na API (chamada async).

Raio de impacto restrito apenas ao Worker (mensagens acumulam, API segue intacta).

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Order System",
    "status": "ready",
    "nodes": [
      {
        "id": "api-pedidos",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "rabbitmq",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "worker-notificacao",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "api-pedidos",
        "target": "rabbitmq",
        "call_type": "async",
        "retries": null,
        "backoff_ms": null,
        "timeout_ms": null
      },
      {
        "source": "rabbitmq",
        "target": "worker-notificacao",
        "call_type": "async",
        "retries": null,
        "backoff_ms": null,
        "timeout_ms": null
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "unavailability",
      "target_node": "worker-notificacao",
      "parameter_value": null,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para api-pedidos",
      "Assumido 1 réplica para rabbitmq",
      "Assumido 1 réplica para worker-notificacao",
      "Assumida capacidade irrestrita para api-pedidos (max_rps_per_replica=null)",

```

```

    "Assumida capacidade irrestrita para rabbitmq (max_rps_per_replica=null)",
    "Assumida capacidade irrestrita para worker-notificacao (max_rps_per_replica=null)",
    "Assumido call_type async para publicação no RabbitMQ",
    "Assumido call_type async para consumo da fila pelo worker"
  ],
  "missing_critical_info": [],
  "reason": null
},
"semantic_issues": [
  "Ponto Único de Falha (SPOF): O componente 'api-pedidos' possui apenas 1 réplica.",
  "Ponto Único de Falha (SPOF): O componente 'rabbitmq' possui apenas 1 réplica.",
  "Ponto Único de Falha (SPOF): O componente 'worker-notificacao' possui apenas 1 réplica."
],
"simulation_results": {
  "scenario": "Falha total e indisponibilidade do componente 'worker-notificacao'",
  "timeouts_exceeded": [
    "O worker/consumidor 'worker-notificacao' está inativo. Mensagens originadas em 'rabbitmq' ficarão
retidas na fila/tópico."
  ],
  "load_amplifications": [],
  "node_saturations": {},
  "blast_radius_impacted_nodes": [
    "worker-notificacao"
  ],
  "availability_impact": "A disponibilidade do fluxo que depende de 'worker-notificacao' cai a zero.",
  "blast_radius_summary": "1 de 3 componentes impactados no sistema (worker-notificacao)"
},
"telemetry": {
  "latency_ms": 1958.27,
  "tokens_in": 1205,
  "tokens_out": 733,
  "total_tokens": 1938,
  "estimated_cost_usd": 0.00129
}
}

```

Response headers

Teste 2.3: Efeito Dominó de Retries

{"architecture_text": "Frontend chama BFF (3 retries). BFF chama Serviço Core (3 retries). Serviço Core consulta Banco de Dados. Tudo síncrono."}
,"question": "O que ocorre se o Serviço Core ficar indisponível?"}

Gabarito Esperado:

Frontend multiplica carga no BFF em 4x.

BFF tenta multiplicar carga no Core em 4x (mas falha).

Carga inútil gerada na rede pela sobreposição de retries aninhados.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Arquitetura Descrita",
    "status": "ready",
    "nodes": [
      {
        "id": "frontend",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "bff",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "service-core",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "database",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "frontend",
        "target": "bff",
        "call_type": "sync",
        "retries": 3,
        "backoff_ms": null,
        "timeout_ms": null
      },
      {
        "source": "bff",
        "target": "service-core",
        "call_type": "sync",
        "retries": 3,
        "backoff_ms": null,
        "timeout_ms": null
      },
      {
        "source": "service-core",
        "target": "database",
        "call_type": "sync",
        "retries": null,
        "backoff_ms": null,

```

```

    "timeout_ms": null
  }
],
"target_availability": 0.999,
"scenario": {
  "scenario_type": "unavailability",
  "target_node": "service-core",
  "parameter_value": null,
  "reason": null
},
"assumptions_made": [
  "Assumido 1 réplica para frontend",
  "Assumido 1 réplica para bff",
  "Assumido 1 réplica para service-core",
  "Assumido 1 réplica para database",
  "Assumida capacidade irrestrita para frontend (max_rps_per_replica=null)",
  "Assumida capacidade irrestrita para bff (max_rps_per_replica=null)",
  "Assumida capacidade irrestrita para service-core (max_rps_per_replica=null)",
  "Assumida capacidade irrestrita para database (max_rps_per_replica=null)"
],
"missing_critical_info": [],
"reason": null
},
"semantic_issues": [
  "Ponto Único de Falha (SPOF): O componente 'frontend' possui apenas 1 réplica.",
  "Ponto Único de Falha (SPOF): O componente 'bff' possui apenas 1 réplica.",
  "Ponto Único de Falha (SPOF): O componente 'service-core' possui apenas 1 réplica.",
  "Ponto Único de Falha (SPOF): O componente 'database' possui apenas 1 réplica.",
  "Risco de Cascata: A chamada síncrona de 'frontend' para 'bff' não possui timeout configurado.",
  "Amplificação de Carga: A chamada 'frontend' -> 'bff' possui 3 retries que podem amplificar falhas.",
  "Risco de Cascata: A chamada síncrona de 'bff' para 'service-core' não possui timeout configurado.",
  "Amplificação de Carga: A chamada 'bff' -> 'service-core' possui 3 retries que podem amplificar falha
s.",
  "Risco de Cascata: A chamada síncrona de 'service-core' para 'database' não possui timeout configurad
o."
],
"simulation_results": {
  "scenario": "Falha total e indisponibilidade do componente 'service-core'",
  "timeouts_exceeded": [
    "A chamada 'bff' -> 'service-core' falha imediatamente (Conexão Recusada / Timeout Imediato)."
  ],
  "load_amplifications": [
    "Sem resposta, os retries do 'bff' amplificam a carga na rede em 4x (160.0 rps inúteis)."
  ],
  "node_saturations": {},
  "blast_radius_impacted_nodes": [
    "bff",
    "service-core"
  ],
  "availability_impact": "A disponibilidade do fluxo que depende de 'service-core' cai a zero.",
  "blast_radius_summary": "2 de 4 componentes impactados no sistema (bff, service-core)"
},
"telemetry": {
  "latency_ms": 2619.79,
  "tokens_in": 1206,
  "tokens_out": 977,
  "total_tokens": 2183,
  "estimated_cost_usd": 0.001483
}
}

```

Response headers

Cenário 3: Multiplicador de Carga (load_multiplier)

Teste 3.1: Black Friday (Saturação Massiva)

{"architecture_text": "O serviço de Carrinho tem 2 réplicas que suportam no máximo 100 rps cada. Atualmente opera em estado estacionário."}
,"question": "Como o sistema reage se a carga de usuários multiplicar por 5?"}

Gabarito Esperado:

Carga simulada excede 200 RPS (capacidade máxima).
Saturação > 100%.
Queda de disponibilidade por gargalo de capacidade.

Response body

Download

```
{
  "status": "sucesso",
  "formal_model": {
    "system_name": "Carrinho",
    "status": "ready",
    "nodes": [
      {
        "id": "carrinho",
        "replicas": 2,
        "max_rps_per_replica": 100,
        "timeout_ms": null
      }
    ],
    "edges": [],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "load_multiplier",
      "target_node": null,
      "parameter_value": 5,
      "reason": null
    },
    "assumptions_made": [],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [],
  "simulation_results": {
    "scenario": "Carga de entrada multiplicada em 5.0x (Pico estimado: 500.0 rps)",
    "node_saturation": {
      "carrinho": "250.0%"
    },
    "availability_impact": "A disponibilidade cai abaixo da meta (99.9%). Gargalo atingido em 'carrinho'. ",
    "blast_radius_impacted_nodes": [
      "carrinho"
    ],
    "timeouts_exceeded": [],
    "load_amplifications": [
      "Tráfego externo aumentou de 100.0 rps para 500.0 rps"
    ],
    "blast_radius_summary": "1 de 1 componentes saturaram com o pico."
  },
  "telemetry": {
    "latency_ms": 1158.54,
    "tokens_in": 1209,
    "tokens_out": 342,
    "total_tokens": 1551,
    "estimated_cost_usd": 0.000983
  }
}
```

Response headers

Teste 3.2: Pico Absorvido

{"architecture_text": "A API de Catálogo roda com 10 réplicas, cada uma suportando 50 rps. A dependência do banco tem capacidade irrestrita."}
,"question": "O que acontece num pico de tráfego de 2x?"}

Gabarito Esperado:

Capacidade total = 500 RPS.

Se assumirmos carga base de 50%, o pico de 2x atinge exatamente 100% de saturação (ou menos).

Disponibilidade se mantém dentro da meta.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Catalog Service",
    "status": "ready",
    "nodes": [
      {
        "id": "catalog-api",
        "replicas": 10,
        "max_rps_per_replica": 50,
        "timeout_ms": null
      },
      {
        "id": "catalog-db",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "catalog-api",
        "target": "catalog-db",
        "call_type": "sync",
        "retries": null,
        "backoff_ms": null,
        "timeout_ms": null
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "load_multiplier",
      "target_node": null,
      "parameter_value": 2,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para catalog-db",
      "Assumida capacidade irrestrita para catalog-db (max_rps_per_replica=null)"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Ponto Único de Falha (SPOF): O componente 'catalog-db' possui apenas 1 réplica.",
    "Risco de Cascata: A chamada síncrona de 'catalog-api' para 'catalog-db' não possui timeout configura
do."
  ],
  "simulation_results": {
    "scenario": "Carga de entrada multiplicada em 2.0x (Pico estimado: 500.0 rps)",
    "node_saturation": {
```

```
    "catalog-api": "100.0%",
    "catalog-db": "625.0%"
  },
  "availability_impact": "A disponibilidade cai abaixo da meta (99.9%). Gargalo atingido em 'catalog-db'.",
  "blast_radius_impacted_nodes": [
    "catalog-api",
    "catalog-db"
  ],
  "timeouts_exceeded": [],
  "load_amplifications": [
    "Tráfego externo aumentou de 250.0 rps para 500.0 rps"
  ],
  "blast_radius_summary": "2 de 2 componentes saturaram com o pico."
},
"telemetry": {
  "latency_ms": 1369.75,
  "tokens_in": 1211,
  "tokens_out": 466,
  "total_tokens": 1677,
  "estimated_cost_usd": 0.001083
}
}
```

Response headers

Teste 3.3: Gargalo no Banco (Assimetria de Escala)

{"architecture_text": "O serviço de Busca escala elasticamente até 20 réplicas (100 rps cada), mas todas leem do mesmo banco primário que suporta no máximo 500 rps no total." , "question": "Qual o impacto de um aumento de 10x no tráfego?"}

Gabarito Esperado:

O serviço de Busca sobrevive (escala).

O banco primário satura violentamente (>100%), derrubando a disponibilidade de todo o fluxo.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "Busca Service",
    "status": "ready",
    "nodes": [
      {
        "id": "busca",
        "replicas": 20,
        "max_rps_per_replica": 100,
        "timeout_ms": null
      },
      {
        "id": "db-primary",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "busca",
        "target": "db-primary",
        "call_type": "sync",
        "retries": null,
        "backoff_ms": null,
        "timeout_ms": null
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "load_multiplier",
      "target_node": null,
      "parameter_value": 10,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para db-primary",
      "Assumida capacidade irrestrita para db-primary (max_rps_per_replica=null)",
      "Assumida chamada sync para leitura do banco"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Ponto Único de Falha (SPOF): O componente 'db-primary' possui apenas 1 réplica.",
    "Risco de Cascata: A chamada síncrona de 'busca' para 'db-primary' não possui timeout configurado."
  ],
  "simulation_results": {
    "scenario": "Carga de entrada multiplicada em 10.0x (Pico estimado: 10000.0 rps)",
    "node_saturation": {
      "busca": "500.0%",
      "db-primary": "12500.0%"
    }
  },
}
```

```
"availability_impact": "A disponibilidade cai abaixo da meta (99.9%). Gargalo atingido em 'db-primary'.",
"blast_radius_impacted_nodes": [
  "busca",
  "db-primary"
],
"timeouts_exceeded": [],
"load_amplifications": [
  "Tráfego externo aumentou de 1000.0 rps para 10000.0 rps"
],
"blast_radius_summary": "2 de 2 componentes saturaram com o pico."
},
"telemetry": {
  "latency_ms": 1748.5,
  "tokens_in": 1218,
  "tokens_out": 626,
  "total_tokens": 1844,
  "estimated_cost_usd": 0.001213
}
}
```

Response headers

Cenário 4: Validações de Borda (Adversariais e Fora de Escopo)

Teste 4.1: Descrição Incompleta (Bloqueante)

```
{"architecture_text": "Temos um sistema em nuvem bem robusto."  
,"question": "O que acontece se a latência subir?"}
```

Gabarito Esperado:

Status: incomplete.

Motor não calcula nada; missing_critical_info é populado exigindo dados dos nós e conexões.

Response body

Download

```
{  
  "status": "recusado_por_falta_de_dados",  
  "mensagem": "A descrição da arquitetura não possui detalhes suficientes para uma simulação matemática p  
recisa. Por favor, complemente sua solicitação.",  
  "informacoes_faltantes": [  
    "Quais são os componentes (serviços, bancos de dados, filas, workers) do sistema?",  
    "Como esses componentes se conectam entre si?",  
    "Existe algum limite de RPS por réplica ou timeout/retries configurados?"  
  ],  
  "telemetry": {  
    "latency_ms": 1793.54,  
    "tokens_in": 1185,  
    "tokens_out": 674,  
    "total_tokens": 1859,  
    "estimated_cost_usd": 0.001232  
  }  
}
```

Response headers

Teste 4.2: Pergunta Fora de Escopo

{"architecture_text": "A API tem 2 réplicas de 50 rps e fala com um banco Postgres síncrono." , "question": "O que acontece se a chave de API do banco vaziar no GitHub?"}

Gabarito Esperado:

Status: recusado_fora_de_escopo (cenário unsupported).

Mensagem: "Não é possível responder com este modelo" (Segurança não é coberta pela matemática de filas).

Response body

Download

```
{
  "status": "recusado_fora_de_escopo",
  "motivo": "Não é possível responder com este modelo, falta de dados sobre impacto de vazamento de chave",
  "telemetry": {
    "latency_ms": 1930.08,
    "tokens_in": 1205,
    "tokens_out": 568,
    "total_tokens": 1773,
    "estimated_cost_usd": 0.00116
  }
}
```

Response headers

Teste 4.3: Texto Contraditório (Engenheiro no Slack)

**{"architecture_text": "O worker consome a fila do Kafka de forma síncrona com timeout HTTP de 5s.",
,"question": "O que acontece se o Kafka cair?"}**

Gabarito Esperado:

O LLM deve resolver a ambiguidade. Uma fila consumida não deve ter timeout HTTP síncrono. O parser deve registrar a correção nas assumptions_made (ex: "Assumido consumo assíncrono para o Kafka, ignorando timeout HTTP") e seguir a simulação de indisponibilidade async do Teste 2.2.

Response body

Download

```
{
  "status": "sucesso_com_alertas",
  "formal_model": {
    "system_name": "KafkaWorkerSystem",
    "status": "ready",
    "nodes": [
      {
        "id": "worker",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      },
      {
        "id": "kafka",
        "replicas": 1,
        "max_rps_per_replica": null,
        "timeout_ms": null
      }
    ],
    "edges": [
      {
        "source": "kafka",
        "target": "worker",
        "call_type": "sync",
        "retries": null,
        "backoff_ms": null,
        "timeout_ms": 5000
      }
    ],
    "target_availability": 0.999,
    "scenario": {
      "scenario_type": "unavailability",
      "target_node": "kafka",
      "parameter_value": null,
      "reason": null
    },
    "assumptions_made": [
      "Assumido 1 réplica para worker",
      "Assumido 1 réplica para kafka",
      "Assumida capacidade irrestrita para worker (max_rps_per_replica=null)",
      "Assumida capacidade irrestrita para kafka (max_rps_per_replica=null)",
      "Assumido call_type sync para consumo da fila Kafka",
      "Assumido timeout HTTP de 5s (5000ms) para a conexão kafka->worker"
    ],
    "missing_critical_info": [],
    "reason": null
  },
  "semantic_issues": [
    "Ponto Único de Falha (SPOF): O componente 'worker' possui apenas 1 réplica.",
    "Ponto Único de Falha (SPOF): O componente 'kafka' possui apenas 1 réplica."
  ],
  "simulation_results": {
    "scenario": "Falha total e indisponibilidade do componente 'kafka'",
    "timeouts_exceeded": [],

```

```
"load_amplifications": [],
"node_saturation": {},
"blast_radius_impacted_nodes": [
  "kafka"
],
"availability_impact": "A disponibilidade do fluxo que depende de 'kafka' cai a zero.",
"blast_radius_summary": "1 de 2 componentes impactados no sistema (kafka)"
},
"telemetry": {
  "latency_ms": 2017.83,
  "tokens_in": 1194,
  "tokens_out": 759,
  "total_tokens": 1953,
  "estimated_cost_usd": 0.001304
}
}
```

Response headers